

MÓDULO I · INGENIERÍA DE IA

02

CAPÍTULO

Cómo aprende una máquina

Cómo aprende una máquina

"Antes de comprender cómo razona un modelo, debemos comprender cómo adquiere conocimiento."

Objetivos de aprendizaje

Al finalizar esta sección serás capaz de:

- Comprender qué significa que un sistema "aprenda".
 - Diferenciar software programado de software entrenado.
 - Entender el concepto de entrenamiento como un proceso de optimización.
 - Incorporar el vocabulario fundamental que se utilizará en el resto del libro.
-

Introducción

En el capítulo anterior desarrollamos una idea central: la Inteligencia Artificial no consiste en herramientas, sino en principios.

Ahora comenzaremos a estudiar el mecanismo que permitió el surgimiento de los sistemas modernos de IA.

La pregunta ya no es **qué** puede hacer un modelo.

La pregunta es **cómo** llega a hacerlo.

Programar versus entrenar

Durante décadas, desarrollar software significó traducir reglas de negocio en código.

El comportamiento del sistema dependía exclusivamente de las instrucciones escritas por los desarrolladores.

En Inteligencia Artificial ocurre algo diferente.

El desarrollador ya no especifica todas las reglas.

Construye un proceso para que el sistema descubra patrones a partir de ejemplos.

Por esa razón hablamos de **entrenamiento** y no simplemente de programación.

¿Qué significa aprender?

Desde una perspectiva ingenieril, aprender significa modificar un modelo para mejorar su desempeño frente a una tarea determinada utilizando experiencia previa.

Esa experiencia normalmente adopta la forma de datos.

El modelo procesa ejemplos, produce resultados, compara esos resultados con el comportamiento esperado y ajusta internamente millones o miles de millones de parámetros.

Este ciclo se repite una y otra vez hasta alcanzar un nivel de desempeño aceptable.

Es importante observar que el modelo no "comprende" en el sentido humano del término.

Optimiza una función matemática para reducir el error cometido durante el entrenamiento.

Una analogía útil

Aprender a lanzar una pelota no consiste en memorizar una única posición del brazo.

Después de cada intento realizamos pequeños ajustes.

Con suficiente práctica, esos ajustes producen un movimiento cada vez más preciso.

El entrenamiento de un modelo sigue una lógica similar.

No memoriza una única respuesta.

Ajusta progresivamente sus parámetros para reducir el error acumulado.

La analogía ayuda a comprender la idea general, aunque el proceso matemático real sea considerablemente más complejo.

Lo que veremos a continuación

En las próximas secciones estudiaremos cómo se representan los datos, qué son los parámetros, cómo se entrenan los modelos modernos y por qué los Large Language Models constituyen una evolución de estos principios.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Los datos: la materia prima del aprendizaje

"Un modelo nunca aprende directamente del mundo. Aprende de la representación del mundo contenida en los datos."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender por qué los datos determinan el límite superior de un modelo.
- Diferenciar dato, característica (*feature*) y etiqueta (*label*).
- Entender la relación entre calidad de datos y calidad del aprendizaje.
- Identificar los principales problemas que aparecen antes del entrenamiento.

Introducción

Cuando hablamos de entrenamiento es fácil imaginar que un modelo "absorbe conocimiento" de manera automática.

En realidad, el proceso depende completamente de los datos disponibles.

Un modelo no observa la realidad.

No comprende el contexto de una organización.

No conoce las intenciones de las personas.

Únicamente procesa ejemplos.

Por ello, la calidad del aprendizaje nunca puede superar la calidad de los datos utilizados.

¿Qué son los datos de entrenamiento?

Los datos de entrenamiento constituyen el conjunto de ejemplos que un algoritmo utiliza para descubrir patrones.

Cada ejemplo representa una observación del problema que deseamos resolver.

Si queremos clasificar correos electrónicos, necesitaremos miles o millones de mensajes previamente clasificados.

Si queremos reconocer objetos en imágenes, necesitaremos imágenes correctamente etiquetadas.

El objetivo no es memorizar ejemplos, sino aprender relaciones que permitan generalizar a situaciones nuevas.

Features y labels

En aprendizaje supervisado aparecen dos conceptos fundamentales.

- **Feature:** información utilizada para describir un ejemplo.
- **Label:** resultado esperado para ese ejemplo.

Supongamos un sistema capaz de detectar correos no deseados.

Las palabras presentes en el mensaje, el remitente o la cantidad de enlaces pueden formar parte de las *features*.

La clasificación "spam" o "no spam" constituye la *label*.

El entrenamiento consiste en aprender la relación entre ambas.

Calidad antes que cantidad

Existe la creencia de que disponer de más datos siempre produce mejores modelos.

La realidad es más matizada.

Duplicados, errores, sesgos, información obsoleta o registros inconsistentes pueden deteriorar significativamente el desempeño.

Por ese motivo, una parte importante del trabajo en proyectos reales consiste en limpiar, validar y gobernar los datos antes del entrenamiento.

Caso real

Una empresa desea predecir qué clientes abandonarán un servicio.

El equipo consigue millones de registros históricos.

Sin embargo, descubre que durante varios años el criterio utilizado para registrar las bajas cambió en distintas sucursales.

Antes de entrenar cualquier modelo es necesario normalizar esa información.

El desafío no es matemático.

Es ingenieril.

Ideas clave

- Los modelos aprenden únicamente de los ejemplos disponibles.
 - La calidad de los datos condiciona la calidad del resultado.
 - Preparar datos suele consumir más esfuerzo que entrenar el modelo.
 - Un arquitecto debe considerar el gobierno de datos como parte de la arquitectura.
-

Próxima sección

Ahora que comprendemos el papel de los datos, estudiaremos cómo una computadora transforma palabras, imágenes o números en representaciones matemáticas capaces de ser procesadas por un modelo.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

De la información a los vectores

"Las computadoras no comprenden palabras, imágenes ni sonidos. Comprenden números."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender por qué toda la información debe representarse numéricamente.
- Entender el concepto de vector como estructura fundamental en IA.
- Relacionar la representación matemática con el aprendizaje automático.
- Prepararte para comprender los embeddings en las próximas secciones.

Introducción

Los seres humanos interpretamos el mundo mediante conceptos.

Leemos una palabra y evocamos una idea.

Observamos una imagen y reconocemos personas, objetos y contextos.

Las computadoras no poseen esa capacidad.

Para un procesador, una palabra no tiene significado hasta que puede transformarse en una representación numérica.

Por ello, antes de que un modelo pueda aprender, toda la información debe convertirse en números.

¿Por qué números?

Los algoritmos de aprendizaje automático operan sobre operaciones matemáticas.

Suman, multiplican, comparan y optimizan valores.

No pueden trabajar directamente con una palabra como **arquitectura** o **software**.

Primero es necesario convertir esos conceptos en una estructura matemática.

Esa estructura recibe el nombre de **vector**.

¿Qué es un vector?

En el contexto de la IA, un vector es una colección ordenada de valores numéricos que representa alguna característica de un objeto.

Ese objeto puede ser:

- una palabra;
- una oración;
- un documento;
- una imagen;
- un sonido;
- un usuario;
- un producto.

Dos objetos similares deberían terminar representados por vectores cercanos entre sí.

Esta idea constituye uno de los principios fundamentales de la IA moderna.

Pensar en un espacio

Imaginemos un mapa.

Cada ciudad ocupa una posición determinada.

La distancia entre dos ciudades permite estimar qué tan cerca se encuentran.

Los espacios vectoriales funcionan de manera similar.

Cada elemento ocupa una posición matemática.

La distancia entre dos vectores expresa el grado de similitud entre los objetos representados.

No significa que "comprendan" el concepto.

Significa que poseen representaciones matemáticas con propiedades comparables.

¿Por qué esto es importante?

Una vez que la información se encuentra representada mediante vectores, los modelos pueden:

- comparar similitudes;
- identificar agrupamientos;
- detectar patrones;
- recuperar información relacionada;
- generar nuevas representaciones.

Sin esta etapa, técnicas modernas como los embeddings, la búsqueda semántica o los Large Language Models serían imposibles.

Caso real

Supongamos un buscador documental dentro de una empresa.

Un usuario consulta:

"licencia por maternidad"

Otro usuario escribe:

"permiso por nacimiento"

Aunque las palabras sean diferentes, ambas consultas deberían conducir a documentos similares.

Esto solo resulta posible si el sistema trabaja sobre representaciones matemáticas del significado y no únicamente sobre coincidencias exactas de texto.

Ideas clave

- Los modelos no procesan conceptos directamente; procesan números.
- Los vectores permiten representar información compleja en forma matemática.
- La cercanía entre vectores expresa similitud, no identidad.
- Esta representación constituye la base de gran parte de la IA moderna.

Próxima sección

Ahora estudiaremos cómo esas representaciones evolucionaron hasta convertirse en los **embeddings**, uno de los conceptos más importantes para comprender los Large Language Models y los sistemas RAG.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Embeddings: representando el significado mediante vectores

"Dos palabras pueden escribirse de forma diferente y, sin embargo, significar casi lo mismo. Los embeddings intentan capturar esa relación en un espacio matemático."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender qué es un embedding y por qué representa un avance respecto de las representaciones tradicionales.
- Entender cómo un modelo transforma información en vectores con significado semántico.
- Diferenciar similitud léxica de similitud semántica.
- Comprender por qué los embeddings son la base de RAG, búsqueda semántica y sistemas modernos de recuperación de información.

Introducción

En la sección anterior vimos que una computadora necesita representar toda la información mediante números.

Sin embargo, representar texto como simples valores numéricos no es suficiente.

Consideremos las siguientes palabras:

- automóvil
- coche
- vehículo

Para una persona resulta evidente que describen conceptos estrechamente relacionados.

Para una computadora que únicamente compara caracteres, son tres palabras completamente distintas.

Durante muchos años este fue uno de los principales obstáculos para construir sistemas capaces de comprender lenguaje natural.

Los embeddings surgieron para reducir esa brecha.

El problema de las representaciones tradicionales

Las primeras técnicas de representación trataban cada palabra como un símbolo independiente.

En ese enfoque:

- "automóvil" no tenía relación matemática con "coche";
- "doctor" era completamente diferente de "médico";
- "programador" no compartía ninguna cercanía con "desarrollador".

El sistema solo podía reconocer coincidencias exactas.

Esto limitaba enormemente la capacidad de búsqueda y comprensión.

¿Qué es un embedding?

Un embedding es una representación vectorial de un objeto que intenta preservar su significado dentro de un espacio matemático.

Ese objeto puede ser:

- una palabra;
- una frase;
- un documento;
- una imagen;
- un fragmento de código;
- un audio.

La idea fundamental consiste en ubicar objetos con significados similares en posiciones cercanas del espacio vectorial.

No importa únicamente cómo están escritos.

Importa lo que representan.

Una analogía

Imaginemos una biblioteca gigantesca.

En lugar de ordenar los libros por orden alfabético, decidimos ubicarlos según su temática.

Los libros sobre bases de datos quedan próximos entre sí.

Los de arquitectura de software aparecen en otro sector.

Los relacionados con aprendizaje automático ocupan otra región.

Si un lector busca información sobre Docker, probablemente también encuentre Kubernetes, contenedores y orquestación cerca de ese lugar.

Los embeddings funcionan de manera parecida.

La proximidad ya no depende de las palabras utilizadas.

Depende del significado aprendido por el modelo.

Espacios semánticos

Cuando millones de vectores se distribuyen en un espacio de alta dimensión comienzan a aparecer agrupamientos naturales.

Conceptos relacionados terminan ocupando regiones cercanas.

Conceptos muy diferentes aparecen alejados.

Aunque un ser humano no pueda visualizar miles de dimensiones, los algoritmos pueden calcular distancias entre esos vectores con enorme precisión.

Estas distancias permiten responder preguntas como:

- ¿Qué documento es más parecido a esta consulta?
- ¿Qué productos presentan características similares?
- ¿Qué respuesta previa se asemeja más a este problema?

¿Cómo se generan?

Los embeddings no se construyen manualmente.

Se aprenden durante el entrenamiento del modelo.

Mientras el algoritmo intenta resolver tareas relacionadas con lenguaje, imágenes o cualquier otro dominio, ajusta progresivamente las posiciones de los vectores.

Con el tiempo, esas posiciones comienzan a reflejar relaciones semánticas presentes en los datos.

Por ese motivo, dos modelos distintos pueden producir embeddings diferentes para el mismo texto.

La calidad dependerá del entrenamiento, de los datos utilizados y del objetivo para el cual fueron diseñados.

Caso real

Supongamos que una empresa posee cien mil documentos internos.

Un empleado realiza la siguiente consulta:

"¿Cómo solicitar licencia por nacimiento?"

Ningún documento contiene exactamente esa frase.

Sin embargo, existen procedimientos titulados:

- Licencia por maternidad.
- Licencia por paternidad.
- Permisos familiares.

Gracias a los embeddings, el sistema puede recuperar esos documentos aunque las palabras utilizadas no coincidan exactamente con la consulta.

La búsqueda deja de ser textual y pasa a ser semántica.

Ideas clave

- Un embedding es un vector que representa significado.
- Objetos similares ocupan posiciones cercanas en el espacio vectorial.
- La similitud deja de depender exclusivamente de coincidencias exactas de palabras.

- Los embeddings constituyen uno de los pilares de la IA moderna.

Próxima sección

Ahora que comprendemos cómo representar significado mediante vectores, estudiaremos cómo los modelos aprenden esas representaciones ajustando millones de parámetros durante el entrenamiento.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Parámetros, pesos y el proceso de entrenamiento

"Un modelo no almacena conocimiento como un libro. Lo distribuye entre millones o miles de millones de parámetros."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender qué son los parámetros de un modelo.
- Diferenciar arquitectura, parámetros y datos de entrenamiento.
- Entender cómo un modelo ajusta sus pesos durante el entrenamiento.
- Comprender por qué entrenar e inferir son procesos diferentes.

Introducción

Cuando se afirma que un modelo posee 7.000 millones o 70.000 millones de parámetros, suele interpretarse como una medida directa de inteligencia.

La realidad es más compleja.

La cantidad de parámetros representa la capacidad del modelo para almacenar relaciones aprendidas, pero no garantiza mejores resultados por sí sola.

Para comprender esta afirmación debemos analizar qué son realmente esos parámetros.

¿Qué es un parámetro?

Un parámetro es un valor numérico interno que participa en los cálculos realizados por el modelo.

Durante el entrenamiento esos valores cambian millones de veces.

Cada modificación intenta reducir el error cometido al resolver una tarea.

Al finalizar el entrenamiento, el conjunto completo de parámetros constituye el conocimiento matemático del modelo.

No almacena frases completas.

No guarda respuestas preparadas.

Almacena relaciones estadísticas distribuidas entre todos sus parámetros.

Arquitectura y parámetros

Es importante no confundir estos conceptos.

La **arquitectura** define cómo está construido el modelo.

Los **parámetros** representan los valores aprendidos por esa arquitectura.

Una analogía útil consiste en pensar en un edificio.

Los planos indican la estructura del edificio.

Los parámetros serían la configuración concreta de cada uno de sus componentes una vez terminada la construcción.

Dos modelos pueden compartir exactamente la misma arquitectura y producir resultados muy distintos si fueron entrenados con datos diferentes.

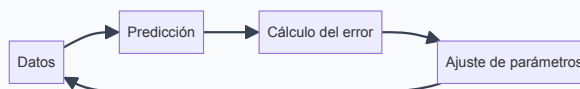
¿Cómo aprende un modelo?

El entrenamiento puede resumirse en un ciclo repetitivo:

1. Recibir un conjunto de ejemplos.
2. Generar una predicción.
3. Compararla con el resultado esperado.
4. Calcular el error.
5. Ajustar los parámetros para reducir ese error.
6. Repetir el proceso millones de veces.

Cada iteración produce cambios muy pequeños.

Sin embargo, la acumulación de esos ajustes permite que el modelo aprenda patrones extremadamente complejos.



Entrenamiento e inferencia

Conviene distinguir claramente dos etapas.

Entrenamiento

- Requiere enormes volúmenes de datos.
- Consume gran capacidad de procesamiento.
- Modifica los parámetros.

Inferencia

- Utiliza un modelo ya entrenado.
- No altera sus parámetros.
- Produce respuestas para nuevos datos de entrada.

La mayoría de los usuarios interactúa únicamente con la etapa de inferencia.

Caso real

Una empresa utiliza un modelo fundacional para responder consultas internas.

No lo entrena desde cero.

En cambio, reutiliza un modelo previamente entrenado y lo complementa con documentación corporativa mediante RAG.

Esta decisión reduce costos y tiempos de implementación sin renunciar a un alto nivel de calidad.

Comprender la diferencia entre entrenar e inferir permite justificar arquitecturas mucho más eficientes.

Ideas clave

- Los parámetros representan el conocimiento aprendido por el modelo.
 - Más parámetros implican mayor capacidad, no necesariamente mejores resultados.
 - Entrenar modifica parámetros; inferir utiliza esos parámetros.
 - La arquitectura y los datos son tan importantes como el tamaño del modelo.
-

Próxima sección

En la siguiente sección estudiaremos cómo surgieron las redes neuronales modernas y por qué el mecanismo de atención (*Attention*) transformó por completo la evolución de los modelos de lenguaje.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Redes neuronales, atención y el nacimiento de los Transformers

"Los grandes modelos de lenguaje no aparecieron por un incremento gradual de potencia de cómputo. Surgieron gracias a un cambio de arquitectura."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender la evolución desde las redes neuronales clásicas hacia los Transformers.
- Entender el concepto de atención (*Attention*) desde una perspectiva ingenieril.
- Identificar por qué esta arquitectura permitió construir los Large Language Models.
- Relacionar estos conceptos con las decisiones arquitectónicas que veremos más adelante.

Introducción

Hasta aquí vimos cómo un modelo aprende a partir de datos y cómo ese aprendizaje queda distribuido entre millones de parámetros.

La siguiente pregunta es inevitable.

¿Cómo puede un modelo procesar textos completos, establecer relaciones entre palabras separadas por cientos de posiciones y generar respuestas coherentes?

Durante muchos años esa fue una de las principales limitaciones de las arquitecturas existentes.

La solución llegó con un cambio conceptual que modificó el rumbo de la Inteligencia Artificial.

Antes de los Transformers

Las primeras redes neuronales aplicadas al procesamiento de lenguaje trabajaban de manera secuencial.

Procesaban una palabra tras otra.

Este enfoque presentaba dos problemas importantes.

El primero era el rendimiento.

La naturaleza secuencial impedía aprovechar completamente el procesamiento paralelo del hardware moderno.

El segundo era la memoria.

A medida que los textos crecían, el modelo tenía dificultades para mantener información relevante sobre elementos que habían aparecido muchas posiciones antes.

En consecuencia, comprender documentos extensos resultaba complejo.

La idea de atención

La innovación consistió en permitir que el modelo evaluara qué partes del texto resultaban relevantes para interpretar cada palabra.

En lugar de recordar únicamente el elemento inmediatamente anterior, el modelo podía asignar distintos niveles de importancia a múltiples posiciones del contexto.

Ese mecanismo recibió el nombre de **atención** (*Attention*).

No implica conciencia ni comprensión.

Representa un procedimiento matemático para ponderar relaciones entre los elementos de una secuencia.

Una analogía

Imaginemos una reunión con veinte participantes.

Mientras una persona habla, no prestamos la misma atención a todos los presentes.

Según el tema, algunas intervenciones adquieren mayor relevancia que otras.

Nuestro foco cambia continuamente.

El mecanismo de atención persigue un objetivo similar.

Permite que el modelo determine qué información merece mayor peso para resolver la tarea actual.

La analogía describe la idea general, aunque el cálculo real sea completamente matemático.

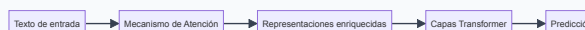
Los Transformers

La arquitectura conocida como **Transformer** incorporó el mecanismo de atención como componente central.

Esto permitió procesar secuencias completas de forma paralela, mejorar el aprovechamiento del hardware y capturar relaciones complejas entre palabras alejadas dentro de un mismo texto.

A partir de este cambio comenzaron a entrenarse modelos con cantidades de datos y parámetros impensadas pocos años antes.

Los Large Language Models modernos descienden directamente de esta arquitectura.



¿Por qué esto cambió la industria?

La arquitectura Transformer no mejoró únicamente el procesamiento del lenguaje.

También demostró ser efectiva para imágenes, audio, código fuente y otros dominios.

Esto impulsó el desarrollo de modelos fundacionales reutilizables para múltiples tareas.

El foco dejó de estar en construir un modelo para cada problema y pasó a adaptarlos mediante diferentes estrategias.

Ideas clave

- Los Transformers sustituyeron limitaciones de arquitecturas anteriores.
- La atención permite ponderar relaciones entre distintos elementos del contexto.
- Los LLM modernos se basan en esta arquitectura.
- Comprender estos principios resulta más importante que memorizar implementaciones específicas.

Próxima sección

En la siguiente sección analizaremos cómo un Transformer entrenado termina convirtiéndose en un **Large Language Model** y por qué estos modelos son capaces de realizar tareas que nunca fueron programadas explícitamente.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Del Transformer al Large Language Model

"Un Transformer es una arquitectura. Un Large Language Model es el resultado de entrenar esa arquitectura con enormes cantidades de datos."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Diferenciar arquitectura de modelo entrenado.
- Comprender qué es el preentrenamiento (*pretraining*).
- Entender el propósito del ajuste fino (*fine-tuning*).

- Explicar por qué un LLM puede resolver tareas para las que nunca fue programado explícitamente.
-

Introducción

En la sección anterior estudiamos la arquitectura Transformer.

Sin embargo, disponer de una buena arquitectura no basta para construir un modelo útil.

Un Transformer recién comienza a adquirir capacidades cuando atraviesa un largo proceso de entrenamiento utilizando cantidades masivas de información.

Ese proceso es el que da origen a un **Large Language Model (LLM)**.

¿Qué significa "Large"?

El término *Large* no hace referencia únicamente a la cantidad de parámetros.

También involucra:

- enormes volúmenes de datos de entrenamiento;
- grandes capacidades de cómputo;
- extensos tiempos de entrenamiento;
- una capacidad creciente para generalizar conocimientos.

El objetivo no es memorizar documentos, sino aprender patrones estadísticos del lenguaje.

El preentrenamiento

Durante el preentrenamiento el modelo procesa cantidades gigantescas de texto.

Su tarea principal suele consistir en predecir cuál debería ser el siguiente token de una secuencia.

Aunque esta tarea parezca simple, obliga al modelo a capturar relaciones gramaticales, semánticas y contextuales.

Con el tiempo desarrolla representaciones internas que luego podrán reutilizarse para una enorme variedad de aplicaciones.

El ajuste fino

Una vez finalizado el preentrenamiento, el modelo puede adaptarse a necesidades concretas.

Este proceso recibe el nombre de **fine-tuning**.

Por ejemplo:

- responder consultas médicas;
- analizar contratos;
- asistir a desarrolladores;
- clasificar documentos legales.

El objetivo ya no es aprender lenguaje general, sino especializar el comportamiento para un dominio determinado.

Capacidades emergentes

Uno de los fenómenos más interesantes de los LLM modernos es la aparición de capacidades que no fueron programadas explícitamente.

Un modelo entrenado para predecir el siguiente token puede:

- resumir textos;
- traducir idiomas;
- generar código;
- responder preguntas;
- explicar conceptos.

Estas capacidades emergen como consecuencia del entrenamiento y de la escala del modelo.

No corresponden a módulos independientes desarrollados por un equipo de ingeniería.

Caso real

Una organización adopta un modelo fundacional para asistir a sus analistas.

Sin modificar la arquitectura, el mismo modelo se utiliza para resumir expedientes, redactar correos, responder preguntas sobre normativa y colaborar con desarrolladores.

La diferencia no reside únicamente en el modelo.

Reside en el contexto, los datos y la forma en que se integra dentro de la arquitectura empresarial.

Ideas clave

- Transformer y LLM no son sinónimos.
 - El preentrenamiento construye conocimiento general.
 - El ajuste fino adapta ese conocimiento a un dominio específico.
 - Muchas capacidades aparecen como resultado de la escala del entrenamiento.
-

Próxima sección

En la siguiente sección analizaremos la **tokenización**, comprenderemos qué es un token y por qué este concepto resulta esencial para entender costos, ventanas de contexto y rendimiento de los modelos modernos.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Tokenización: cómo leen los modelos de lenguaje

"Un modelo de lenguaje no procesa palabras ni caracteres. Procesa tokens."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender qué es un token y por qué es la unidad básica de procesamiento de un LLM.
 - Diferenciar caracteres, palabras y tokens.
 - Entender la relación entre tokenización, ventana de contexto y costo.
 - Identificar el impacto arquitectónico de la cantidad de tokens.
-

Introducción

Cuando un usuario escribe una consulta en un chatbot, suele pensar que el modelo lee exactamente las mismas palabras que aparecen en pantalla.

No ocurre así.

Antes de que el modelo pueda procesar un texto, este debe transformarse en una secuencia de unidades llamadas **tokens**.

La tokenización constituye el primer paso de prácticamente todos los modelos modernos de lenguaje.

¿Qué es un token?

Un token es una unidad de información utilizada por el modelo durante el procesamiento.

Dependiendo del algoritmo de tokenización, un token puede representar:

- una palabra completa;
- parte de una palabra;
- un signo de puntuación;
- un número;
- un espacio;
- un carácter especial.

Por ejemplo, la palabra **internacionalización** podría dividirse en varios tokens, mientras que palabras muy frecuentes podrían corresponder a un único token.

Por este motivo no existe una equivalencia fija entre palabras y tokens.

¿Por qué no utilizar palabras?

Si cada idioma tuviera un vocabulario limitado y estable, trabajar con palabras sería suficiente.

Sin embargo, el lenguaje natural presenta desafíos importantes:

- aparecen palabras nuevas constantemente;
- existen nombres propios y términos técnicos;
- distintos idiomas comparten fragmentos comunes;
- una misma raíz puede generar muchas variantes.

Dividir el texto en fragmentos reutilizables permite representar de manera más eficiente un vocabulario prácticamente ilimitado.

Del texto a los números

Una vez tokenizado el texto, cada token recibe un identificador numérico.

Posteriormente esos identificadores se transforman en embeddings y continúan recorriendo las distintas capas del modelo.

El proceso puede resumirse así:



Aunque para el usuario todo sucede en una fracción de segundo, internamente el modelo nunca trabaja directamente con palabras.

Tokens y ventana de contexto

La tokenización también determina cuánto contenido puede procesar un modelo simultáneamente.

La **ventana de contexto** define el número máximo de tokens que el modelo puede considerar durante una interacción.

Dentro de ese límite se incluyen:

- el mensaje del usuario;
- las instrucciones del sistema;
- el historial de la conversación;
- documentos adicionales;
- la respuesta que generará el modelo.

Cuando el límite se alcanza, parte del contexto debe descartarse o resumirse.

Impacto en costos y rendimiento

En la mayoría de los servicios comerciales el consumo se mide en tokens.

Esto tiene consecuencias directas para la arquitectura.

Un sistema que envía documentos completos en cada consulta consumirá muchos más recursos que otro que recupera únicamente los fragmentos relevantes.

Por esta razón, comprender la tokenización resulta esencial para diseñar soluciones eficientes, especialmente cuando se utilizan técnicas como RAG.

Caso real

Una organización incorpora un asistente documental.

En la primera versión, cada consulta envía manuales completos de cientos de páginas al modelo.

Los costos aumentan rápidamente y la latencia se vuelve inaceptable.

Al reemplazar esa estrategia por una búsqueda semántica que recupera únicamente los fragmentos relevantes, la cantidad de tokens disminuye de forma significativa, mejorando tanto el rendimiento como el costo operativo.

El cambio no consistió en utilizar un modelo diferente.

Consistió en diseñar mejor la arquitectura.

Ideas clave

- Los modelos procesan tokens, no palabras.
 - La cantidad de tokens determina el tamaño del contexto disponible.
 - El consumo de tokens influye directamente en costos y rendimiento.
 - Comprender la tokenización permite tomar mejores decisiones arquitectónicas.
-

Próxima sección

En la siguiente sección estudiaremos la **ventana de contexto**, sus limitaciones y por qué este concepto impulsó el desarrollo de arquitecturas como Retrieval-Augmented Generation (RAG).

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

La ventana de contexto: el límite de la memoria inmediata

"Un modelo puede haber aprendido sobre millones de documentos y, sin embargo, ser incapaz de utilizar un documento que no entra en su ventana de contexto."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender qué es la ventana de contexto (*Context Window*).
 - Diferenciar conocimiento entrenado de contexto suministrado durante la inferencia.
 - Entender cómo el tamaño del contexto afecta la calidad, el rendimiento y el costo.
 - Comprender por qué esta limitación impulsó el desarrollo de arquitecturas como RAG.
-

Introducción

Una de las preguntas más frecuentes al comenzar a trabajar con modelos de lenguaje es:

"¿Por qué no puedo enviar toda la documentación de mi empresa en una única consulta?"

La respuesta está relacionada con uno de los conceptos más importantes de los Large Language Models: la **ventana de contexto**.

Aunque un modelo haya sido entrenado con enormes cantidades de información, durante cada interacción solo puede procesar una cantidad limitada de tokens.

Comprender esta restricción resulta esencial para diseñar aplicaciones eficientes.

¿Qué es la ventana de contexto?

La ventana de contexto representa la cantidad máxima de información que un modelo puede considerar simultáneamente para generar una respuesta.

Dentro de esa ventana conviven distintos elementos:

- las instrucciones del sistema;
- el historial de la conversación;
- la consulta del usuario;
- documentos adicionales;
- herramientas utilizadas por el modelo;

- la respuesta que está generando.

Todos ellos compiten por el mismo espacio.

Conocimiento permanente versus memoria temporal

Es importante distinguir dos conceptos.

El **conocimiento entrenado** forma parte de los parámetros del modelo y permanece constante durante la inferencia.

El **contexto**, en cambio, existe únicamente durante la conversación actual.

Una analogía útil consiste en comparar un profesional con una reunión de trabajo.

La experiencia acumulada durante años representa su conocimiento permanente.

Los documentos distribuidos al comienzo de la reunión constituyen el contexto temporal utilizado para resolver una tarea específica.

Cuando la reunión termina, esos documentos dejan de formar parte del contexto inmediato.

¿Qué ocurre cuando el contexto es demasiado grande?

Cuando la información supera el tamaño permitido aparecen distintas alternativas:

- eliminar parte del historial;
- resumir conversaciones anteriores;
- dividir documentos en fragmentos;
- recuperar únicamente la información relevante.

Las últimas dos estrategias dieron origen a arquitecturas modernas como **Retrieval-Augmented Generation (RAG)**.

En lugar de enviar toda la base documental, el sistema recupera solo los fragmentos más relacionados con la consulta.

Arquitectura versus modelo

En muchos proyectos el problema no consiste en disponer de un modelo más grande.

Consiste en enviar información innecesaria.

Un diseño arquitectónico adecuado suele producir mejores resultados que incrementar indiscriminadamente el tamaño del contexto.

Esto explica por qué sistemas relativamente pequeños pueden superar a soluciones mucho más costosas cuando la recuperación de información está correctamente diseñada.

Caso real

Una empresa desea consultar un repositorio con veinte mil procedimientos internos.

La primera propuesta consiste en incluir todos los documentos en cada consulta.

Los costos y la latencia resultan inaceptables.

La solución finalmente implementada utiliza embeddings para localizar los cinco documentos más relevantes y solo esos fragmentos se incorporan al contexto del modelo.

El resultado mejora simultáneamente la precisión, el rendimiento y el costo operativo.

Ideas clave

- La ventana de contexto limita la cantidad de información procesable en una interacción.
- El conocimiento entrenado y el contexto son conceptos diferentes.
- Un mayor contexto no siempre implica mejores respuestas.
- La arquitectura determina qué información debe enviarse realmente al modelo.

Próxima sección

En la última sección del capítulo integraremos todos los conceptos estudiados y construiremos una visión completa del funcionamiento interno de un Large Language Model, preparando el terreno para los capítulos dedicados a RAG y arquitectura de sistemas inteligentes.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Integrando los conceptos: cómo piensa un Large Language Model

"Comprender un modelo no significa conocer cada una de sus ecuaciones. Significa entender cómo interactúan sus componentes para producir un comportamiento útil."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Integrar los conceptos desarrollados a lo largo del capítulo.
- Explicar, a alto nivel, el recorrido completo desde un texto de entrada hasta una respuesta generada.
- Comprender qué elementos pertenecen al modelo y cuáles a la arquitectura que lo rodea.
- Prepararte para estudiar Retrieval-Augmented Generation (RAG) en el próximo capítulo.

Introducción

En este capítulo recorrimos el funcionamiento interno de un modelo moderno de lenguaje.

Cada concepto estudiado constituye una pieza de un sistema mucho mayor.

Analizados de forma aislada pueden parecer independientes.

Sin embargo, cuando se integran describen el flujo completo de funcionamiento de un Large Language Model.

El recorrido de una consulta

Cuando un usuario envía una pregunta, el proceso puede resumirse de la siguiente manera:

1. El texto se tokeniza.
2. Cada token se transforma en una representación numérica.
3. Esa representación se convierte en embeddings.
4. El Transformer procesa la secuencia utilizando mecanismos de atención.
5. Los parámetros aprendidos durante el entrenamiento permiten estimar el siguiente token más probable.
6. El proceso se repite hasta construir la respuesta completa.

Aunque internamente intervienen miles de millones de operaciones matemáticas, esta secuencia resume el comportamiento general del sistema.



Lo que el modelo sabe y lo que no sabe

A lo largo del capítulo diferenciamos claramente dos fuentes de conocimiento.

Por un lado, el conocimiento adquirido durante el entrenamiento y almacenado en los parámetros del modelo.

Por otro, el contexto suministrado durante cada interacción.

Esta diferencia resulta fundamental.

Un modelo no consulta Internet por sí mismo.

No accede automáticamente a la base documental de una empresa.

Solo puede utilizar la información presente en sus parámetros y en el contexto recibido durante la inferencia.

Cuando esto no alcanza, es necesario incorporar componentes externos.

Aquí comienza el terreno de la arquitectura.

Del modelo al sistema

Uno de los errores más frecuentes consiste en identificar un producto basado en IA únicamente con el modelo de lenguaje.

En realidad, el modelo representa solo uno de sus componentes.

Un sistema preparado para producción suele incluir además:

- autenticación;
- bases de datos;
- almacenamiento documental;
- observabilidad;
- monitoreo;
- registro de auditoría;
- mecanismos de recuperación de información;
- validaciones de negocio;
- integración con APIs.

La calidad final depende de la interacción entre todos estos elementos.

Lo que aprenderemos a continuación

Hasta ahora analizamos el funcionamiento interno del modelo.

Sin embargo, la mayoría de las aplicaciones empresariales necesita trabajar con información que no formó parte del entrenamiento.

Normativa interna.

Procedimientos.

Contratos.

Historias clínicas.

Expedientes.

Catálogos.

Bases de conocimiento.

¿Cómo puede un modelo responder utilizando esa información sin volver a entrenarse?

La respuesta conduce directamente al siguiente gran concepto del libro: **Retrieval-Augmented Generation (RAG)**.

Resumen del capítulo

Durante este capítulo aprendimos que:

- un modelo aprende ajustando parámetros;
- los datos constituyen la base del aprendizaje;
- la información debe transformarse en vectores;
- los embeddings representan significado;
- los Transformers permiten modelar relaciones complejas mediante atención;
- los LLM son el resultado de entrenar esas arquitecturas a gran escala;
- los modelos procesan tokens dentro de una ventana de contexto limitada.

Estos conceptos serán reutilizados constantemente en el resto del libro.

Mensaje final

Comprender cómo funciona un Large Language Model permite abandonar la visión de "caja negra".

A partir de ahora podremos analizar arquitecturas completas y justificar técnicamente decisiones que van mucho más allá de elegir un proveedor o escribir un prompt.

Ese es el verdadero objetivo de la Ingeniería de IA.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

